

TCP Reliable Data Transfer

(Sender Logic, Fast Retransmit, and Error Recovery Policies)

1. Reliable Data Transfer Principles

TCP creates a **reliable data transfer service** on top of IP's unreliable "best-effort" service. It ensures that the byte stream read by the receiving process is:

- **Uncorrupted** (Integrity check).
- **Without Gaps** (No missing data).
- **Without Duplication** (Handling retransmissions).
- **In Sequence** (Ordering bytes correctly).

Single Timer Recommendation

While earlier rdt protocols used a timer for every segment, the recommended TCP implementation [RFC 6298] uses **only a single retransmission timer**, even if multiple segments are in flight.

2. TCP Sender Logic (Simplified)

The simplified TCP sender handles three major events:

1. **Data Received from App Above:** Creates a segment with `seq = NextSeqNum`, starts the timer if not running, and increments `NextSeqNum`.
2. **Timeout:** Retransmits the **oldest unacknowledged segment** and restarts the timer.
3. **ACK Received:** Checks if the ACK value y is greater than the `SendBase`. If so, updates `SendBase` and restarts the timer if other segments are still unacknowledged.

Pseudo-code: Simplified TCP Sender

```
NextSeqNum = InitialSeqNumber
SendBase = InitialSeqNumber
loop (forever) {
    switch(event)
        event: data received from application above
            create TCP segment with sequence number NextSeqNum
            if (timer not running) start timer
            pass segment to IP; NextSeqNum += length(data)
        event: timeout
            retransmit oldest unacknowledged segment; start timer
        event: ACK received with value y
            if (y > SendBase) {
                SendBase = y
                if (unacknowledged segments exist) start timer
            }
}
```

3. Retransmission Scenarios

3.1 Scenario 1: Lost Acknowledgment

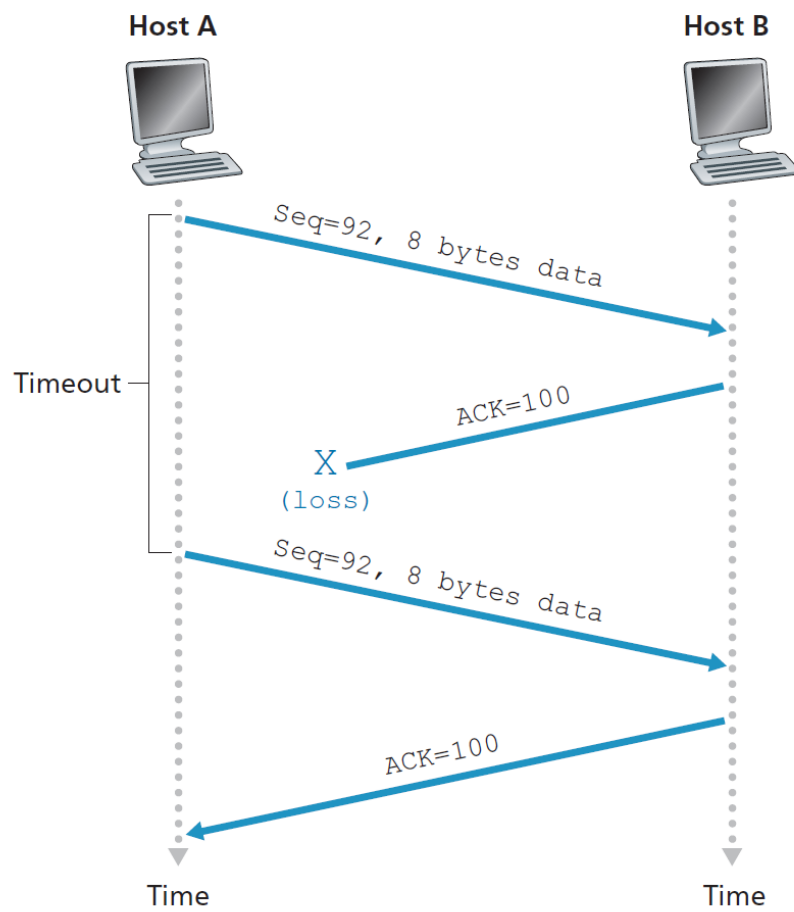


Figure 1: Retransmission due to a Lost Acknowledgment

If Host A sends a segment (Seq=92, 8 bytes) and the ACK (ACK=100) is lost, a timeout occurs. Host A retransmits. Host B receives the duplicate, recognizes it by the sequence number, and discards the data but **resends the ACK**.

3.2 Scenario 2: Delayed Acknowledgments

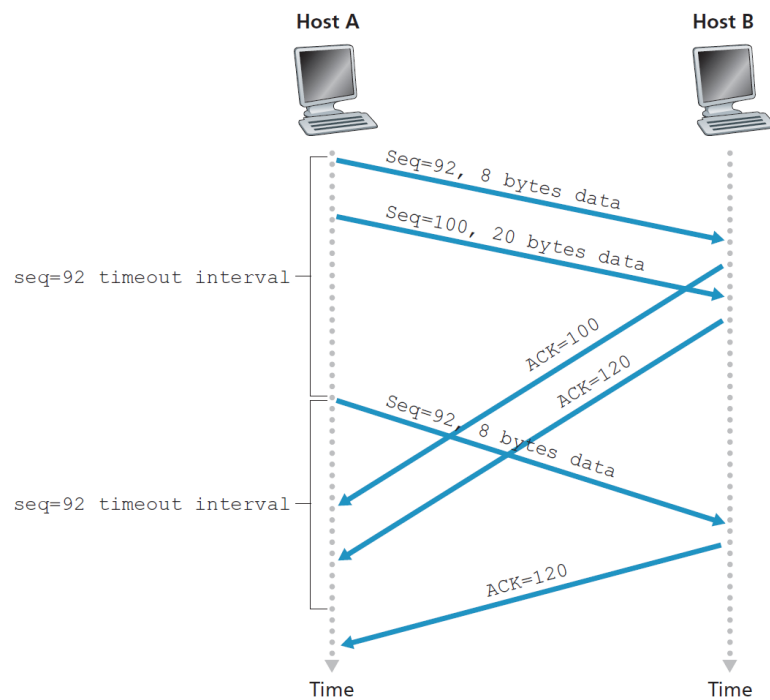


Figure 2: Segment 100 Not Retransmitted (Delayed ACKs)

If Host A sends two segments back-to-back (Seq=92 and Seq=100), and both arrive but their ACKs are delayed. If the ACK for the second segment (ACK=120) arrives **before** the timeout for the first, the first segment is **not** retransmitted because of **cumulative ACKs**.

3.3 Scenario 3: Cumulative ACK Benefit

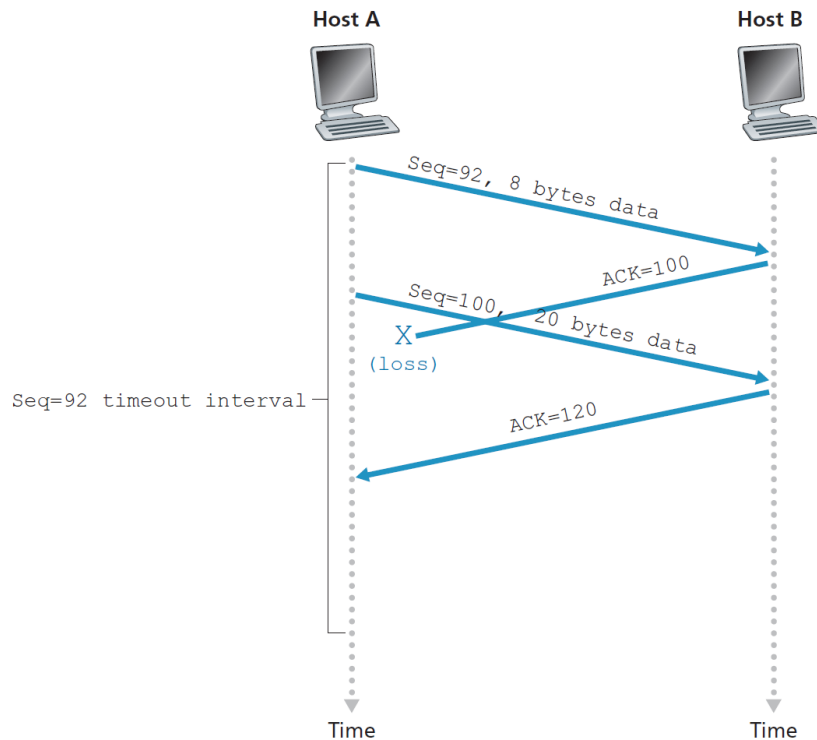


Figure 3: Cumulative Acknowledgment Avoids unnecessary Retransmission

Suppose ACK=100 is lost, but ACK=120 arrives before the timeout. Host A knows the receiver got everything up to byte 119 and thus ignores the missing ACK=100.

4. Doubling the Timeout Interval

Whenever a timeout occurs, TCP retransmits the missing segment and **doubles** the TimeoutInterval.

- This acts as a **limited form of congestion control**.
- Congestion usually causes drops; persistent retransmissions would worsen it.
- The interval is reset to normal (calculated via RTT) only after data is received from the app or a valid ACK is received.

5. Fast Retransmit

Timeouts can be long, causing delays. **Fast Retransmit** allows the sender to detect loss early using **Duplicate ACKs**.

What is a Duplicate ACK?

A duplicate ACK re-acknowledges a segment for which an earlier ACK was already received. It typically indicates a **gap** (an out-of-order segment arrived).

The 3-Duplicate ACK Rule

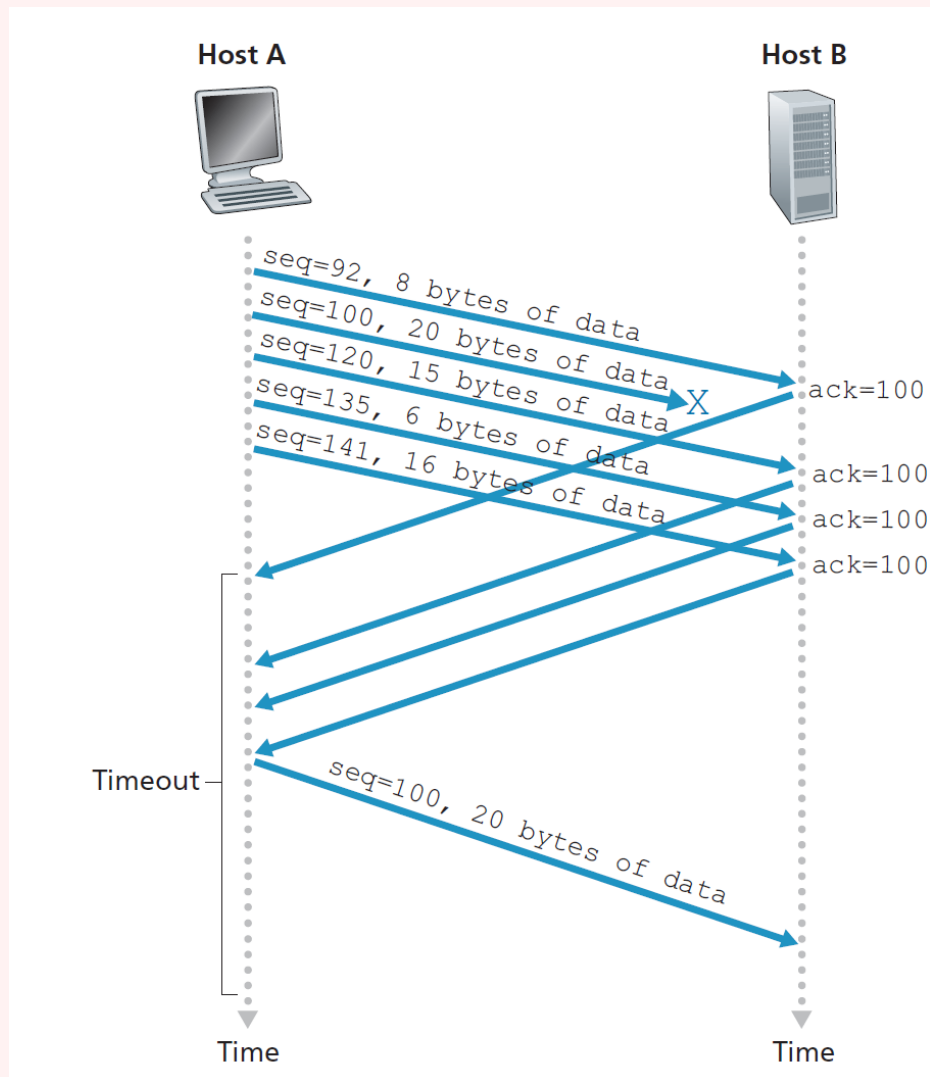


Figure 4: Fast Retransmit Example

If the TCP sender receives **3 duplicate ACKs** for the same data, it assumes the next segment has been lost and retransmits it ****immediately****, before the timer expires.

6. TCP ACK Generation Policy

Receiver ACK recommendations (RFC 5681)

Event	Receiver Action
In-order segment arrived. All data up to expected Seq acknowledged.	Delayed ACK. Wait 500ms for next segment. If none, send ACK.
In-order segment arrived. One other in-order segment waiting for ACK.	Cumulative ACK. Immediately send ACK for both.
Out-of-order segment (Seq higher than expected). Gap detected.	Duplicate ACK. Immediately resend ACK for last in-order byte.
Arrival of segment that fills (or partially fills) a gap.	Immediate ACK. (If it starts at lower end of gap).

7. The Hybrid Nature: GBN or SR?

Is TCP Go-Back-N or Selective Repeat?

- **Like GBN:** It uses cumulative ACKs and maintains only the `SendBase`.
- **Like SR:** Many implementations **buffer out-of-order segments** and retransmit **only the missing segment** (rather than everything after it).

Selective Acknowledgment (SACK)

[RFC 2018] allows a receiver to explicitly ACK out-of-order blocks, making TCP even more like Selective Repeat.

8. Summary

Reliable Transfer Highlights

- TCP handles lost, corrupted, or out-of-order IP datagrams.
- Uses a single timer for efficiency.
- Cumulative ACKs enable recovery even if individual ACKs are lost.
- Fast Retransmit avoids long timeout delays.
- It is a ****hybrid**** of GBN and SR features.